

RISA-bot Competition Workshop

Day 1 Presenter's Guide & Script

*A slide-by-slide presentation script, interactive live demos,
hands-on student activity guidelines, and parameter tuning references
for Linux recap, ROS 2 topics, image processing pipelines, and LiDAR wall followers.*

Developed by the RISA-bot Core Team

Version 1.0 - July 2026

1. Core Technical Terms Explained for Students

Use these definitions and analogies during the presentation to help students grasp the core math and computer science concepts behind RISA-bot.

Student Explanation: CLAHE (Contrast Limited Adaptive Histogram Equalization)

Definition: An image processing technique that enhances image contrast locally in small sections rather than the whole image at once.

Analogy for Students: CLAHE is like a smart pair of sunglasses that automatically adjusts its tint depending on where you look. If you look at a bright sky, it dims just that spot; if you look down into a dark shadow, it brightens only the shadow so you can see clearly. Unlike standard filters, it divides the image into a grid of small tiles and optimizes each tile individually. In flat areas like a plain white wall, it limits the contrast boost to prevent amplifying tiny camera noise.

Student Explanation: PID Controller (Proportional-Integral-Derivative)

Definition: A feedback control loop that calculates steering adjustments based on three views: Present Error (P), Past Error History (I), and Future Rate of Change (D).

Analogy for Students: Think of driving a car on a highway. P (Proportional) is looking at the Present: how far are you from the center line right now? If slightly off, you turn slightly. If way off, you turn hard. (If you only use P, you will overshoot and sway left-right forever). D (Derivative) looks at the Future: how fast are you rushing back to the center? If heading back too fast, D dampens your steering by turning the wheel straight BEFORE you cross the line. I (Integral) looks at the Past: are you drifting over time? If your wheels are misaligned, I accumulates that error and pushes you back to center.

Student Explanation: Kalman Filter

Definition: An algorithm that estimates the true state/position of a system by blending noisy physical measurements with a mathematical prediction model.

Analogy for Students: Imagine walking in a dark room. You know your stride is 0.8 meters, so you count steps to estimate your position (Prediction). You also check a glitchy GPS on your wrist that jumps around (Measurement). A Kalman filter mathematically combines both: it knows the reliability of your steps and the noise of the GPS, and calculates the most accurate middle ground. On RISA-bot, it prevents camera lane detections from jumping due to floor glare.

Student Explanation: Hysteresis

Definition: A system behavior where the transition threshold is padded with a historical buffer to prevent rapid on/off flickering.

Analogy for Students: Think of a home AC. If it turned on the exact microsecond the temperature hit 22.01 C, and off at 21.99 C, it would click on and off every two seconds and break. Instead, it turns on at 23 C and off at 21 C. That gap is the hysteresis buffer. On RISA-bot, it prevents flickering between camera (LANE_FOLLOW) and LiDAR (TUNNEL) modes at the entrance by requiring 3 consecutive frames of wall visibility to switch.

Student Explanation: EMA (Exponential Moving Average)

Definition: A smoothing average that gives more weight to recent data points while slowly forgetting older ones.

Analogy for Students: Your mood today is mostly affected by what happened today (80% weight), but you still carry a little of how you felt yesterday (20% weight). EMA filters out sudden, single bad inputs (like a brief gap in the wall) without lagging behind. In RISA-bot, it smooths LiDAR steering commands so the servo wheels don't jerk.

Student Explanation: HSV Color Space (Hue, Saturation, Value)

Definition: A color representation that separates color shade/tint (Hue) from purity (Saturation) and brightness (Value).

Analogy for Students: RGB is like mixing colored light bulbs, which changes drastically under different room lights. HSV is how humans pick paint: Hue (what color shade is it?), Saturation (how vivid or faded is it?), and Value (how bright is the light shining on it?). For traffic lights, we filter only by Hue, meaning the robot is not blinded by shadows or bright glare.

2. Presentation Flow & Slide Script (Day 1)

This section details the talking points, visual layout descriptions, and instructions for each block of Day 1's 64-slide deck.

Block 1: ROS & Linux Recap (Slides 1-13)

Slide 1-5: Welcome & Workshop Intro

- Visuals: Dark title slide with RISA-bot chassis, schedule, and course map.
- Talking Points: Welcome everyone! Over the next two days, we will learn how RISA-bot solves complex competition challenges on standard accessible hardware. Today is Day 1: we recap Linux/ROS, inspect architecture, tour the dashboard, and calibrate lane following and tunnel navigation.
- Rotation Details: We have 21 people split into 7 groups. During track testing, you will rotate in pairs (Slot A: Groups 1-2, Slot B: Groups 3-4, Slot C: Groups 5-6, Slot D: Group 7).

Slide 6-13: ROS & Linux Recap

- Visuals: Terminal commands list, SSH syntax, and workspace commands.
- Talking Points: Let's make sure we can talk to the robot. SSH into your robot: 'ssh sunrise@<robot_ip>'. Go to the workspace: 'cd ~/risabotcar_ws'. Use the alias 'sos' to source the workspace, and 'cb' to build it. To start the robot, run:

```
ros2 launch risabot_automode bringup.launch.py
```

- Talking Points (cont.): This starts hardware nodes. In a second terminal, inspect the topics and parameters:

```
ros2 topic list
ros2 topic echo /lane_error
ros2 param get /auto_driver forward_speed
```

- Student Activity (TRY IT): SSH into the robot, launch the bringup, and echo the lane error topic to confirm connection.

Block 2: RISA-bot Overview & Goals (Slides 14-22)

Slide 14-22: Course Layout & SSH Setup

- Visuals: 6.4m x 4m course dimensions, Lap 1 challenges layout (obstruction, roundabout, tunnel, hill, bumper, traffic light) and Lap 2 route.
- Talking Points: RISA-bot uses a single state-machine brain that monitors sensors and activates specific algorithms. We use NoMachine to get graphical desktop access. Connect on port 4000. Open rqt_image_view to check the camera feed:

```
ros2 run rqt_image_view rqt_image_view
```

- Talking Points (cont.): Your gamepad joystick has a Start button for autonomous control and a Back button for manual safety override. Run 'jstest /dev/input/js0' to verify button mappings.
- Student Activity (TRY IT): Set up the NoMachine connection, check joystick inputs, and verify the raw camera video feed.

Block 3: Architecture & Component Setup (Slides 23-30)

Slide 23-30: Hardware-Perception-Brain Flow

- Visuals: Architecture flow diagram: Sensors -> Perception -> Brain (auto_driver) -> Motors.
- Talking Points: Our code uses a staggered startup: hardware starts at 0s, perception nodes at 3s, and auto_driver at 5s. A single file config/params.yaml holds all tunable values. Let's practice setting a parameter live:

```
ros2 param set /auto_driver forward_speed 0.12
ros2 param get /auto_driver forward_speed
```

- Student Activity (TRY IT): Inspect bringup.launch.py and params.yaml. Practice modifying the forward speed parameter live in the terminal.

Block 4: Dashboard Tour (Slides 31-37)

Slide 31-37: Web Interface & Param Setters

- Visuals: Slide 31: 'MODULE C' transition slide. Dashboard tour highlights: camera tabs, state display, LiDAR canvas, and parameter drawers.
- Talking Points: Open 'http://<robot_ip>:8080' in your web browser. You can monitor the state (LANE_FOLLOW, MANUAL, TUNNEL), view camera debug overlays, and slide open the parameter drawer to modify settings. Always click the Save button (Save icon) to write parameters permanently.
- Student Activity (TRY IT): Log into the web dashboard, open the Lane Lines overlay, change forward_speed to 0.20, check the result, and click Save.

Block 5: Lane Detection Pipeline (Slides 38-44)

Slide 38-44: Image Processing & Calibrations

- Visuals: Image processing sequence chart. Crop, CLAHE, Binary, Morphology, scanlines, Kalman, lane error.
- Talking Points: The camera processes 320x240 frames. We crop the top half using 'crop_ratio_base' to ignore ceilings. CLAHE boosts contrast. Invert binary thresholding isolates the black tape. Scanlines detect left (blue) and right (pink) edges. Kalman filters remove frame-to-frame noise.
- Talking Points (cont.): The parameter 'crop_ratio_base' adjusts looking range, and 'white_threshold' controls sensitivity. Tune these first when lighting shifts!

```
ros2 param set /line_follower_camera crop_ratio_base 0.55
ros2 param set /line_follower_camera white_threshold 100
```

- Student Activity (TRY IT): Slide the robot over the tape. Tune crop ratio and white threshold until blue/pink dots cleanly align with the tape edges on the dashboard. Save settings.

Block 6: PID Steering (Slides 45-50)

Slide 45-50: Steering Control & Live Calibration

- Visuals: PID steering formula. Symptom-fix diagnostics table. Starting parameters code block.
- Talking Points: We convert lane error to steering commands using PID. pid_kp is proportional turn strength. pid_kd is the damping. pid_ki handles drift. In sharp corners, speed is scaled down automatically. Set safe starting values:

```
ros2 param set /auto_driver forward_speed 0.10
ros2 param set /auto_driver pid_kp 0.5
ros2 param set /auto_driver pid_kd 0.2
```

- Talking Points (cont.): If the robot oscillates on straight lines, lower Kp and raise Kd. If it cuts corners, raise crop ratio. Only speed up once steering is stable.
- Student Activity (TRY IT): Place the robot on Lane 1. Press Start. Calibrate Kp and Kd based on path behavior. Save parameters.

Block 7: Tunnel Navigation (Slides 51-60)

Slide 51-60: LiDAR Wall Centering & Switch Logic

- Visuals: LiDAR wall sectors (Left: 15 to 120, Right: -120 to -15 degrees). Hysteresis state transition chart.
- Talking Points: Inside tunnels, cameras fail. LiDAR fires laser scans. The robot classifies left and right wall points, calculates lateral error and heading curves, smooths it via EMA, and centers itself. We require 3 frames of wall confirmation (hysteresis) to switch in and out of tunnel mode. Tune these commands:

```
ros2 param set /tunnel_wall_follower kp 3.0
ros2 param set /tunnel_wall_follower kd 0.8
ros2 topic echo /tunnel_debug
```

- Student Activity (TRY IT): Place the robot in the tunnel. Monitor /tunnel_debug. Run autonomous mode and verify that the robot auto-switches to LiDAR centering and exits back to camera lane following.

Block 8: Full Day 1 Tuning Run (Slides 61-64)

Slide 61-64: End-to-End Optimizations

- Visuals: Calibration order list. Parameter quick cheat sheet. Supervisor diagnostics table.
- Talking Points: Today's final step is a full run. Remember the golden calibration order: white threshold -> crop ratio -> PID steering -> tunnel wall following -> speed. Change only one value at a time. Save your final configuration before leaving today!
- Student Activity (TRY IT): Execute end-to-end autonomous runs on Lane 1. Tune parameters to optimize speed while keeping the robot centered. Save all configurations.

3. Parameters & Troubleshooting Reference Tables

Key Parameter Reference Table

Parameter Name	Node Path	Default	Calibrating Action
forward_speed	/auto_driver	0.15	Base driving speed (m/s). Keep low during tuning.
pid_kp	/auto_driver	0.8	Steering strength. Lower if oscillating; raise if lazy.
pid_kd	/auto_driver	0.20	Steering damping. Raise if wobbling after curves.
pid_ki	/auto_driver	0.01	Drift correction. Keep small to avoid wind-up.
white_threshold	/line_follower_camera	100	Binary threshold. Calibrate first to match lighting.
crop_ratio_base	/line_follower_camera	0.55	Crop line. Lower to see closer; raise to look ahead.
kp	/tunnel_wall_follower	5.0	Wall centering strength. Lower if bouncing between walls.
kd	/tunnel_wall_follower	0.5	Wall follower damping. Raise to stabilize centering.

Symptom & Diagnostic Troubleshooting Matrix

Observed Symptom	Probable Cause	Corrective Fix
Shakes left-right rapidly on straights	Steering proportional gain too high	Lower pid_kp to 0.4 - 0.5, raise pid_kd to 0.3
Cuts inside corners and hits tape	Looking too far ahead; turning too early	Decrease crop_ratio_base (try 0.3 - 0.4)
Loses lane or fails to steer in curves	Looking too close, or Kp gain too low	Increase crop_ratio_base (0.5) or raise pid_kp
Jumps/ignores lines on dashboard	Glare under spotlight blinds camera	Adjust white_threshold up/down (try 150 or 80)
Bounces between walls in tunnel	LiDAR centering gain is too high	Lower tunnel kp to 3.0 and raise kd to 0.8
Flickers at tunnel entry and exit	LiDAR points count below threshold	Lower min_wall_points to 3 to ease trigger
Drifts or hugs one side of the tunnel	LiDAR center position misaligned	Adjust target_center_dist by +/- 0.02